NBA Injury Risk Prediction

Jeremy Tang, Ashhad Jaffer, Naseem Heydari

The NBA has evolved into a sport that has been dominated by astounding athleticism: Modern players today combine speed, size, and explosiveness into their play that has never been seen before in the previous generations of league play. With these jumps in abilities comes an increased physical burden on players, where games have intensified accordingly. Across an 82-game regular season, teams play three to five games per week, often on consecutive nights, placing substantial strain on player bodies. Load management has arisen from this increased physical burden, where teams deliberately rest healthy star players to reduce injury risk. This speaks directly to how impactful injury risks are for teams, and how injury prevention has become a competitive strategy for winning. We're interested in this phenomenon and used this project to explore whether player-level statistics, physical attributes, and injury history can be used to predict a player's likelihood of appearing on injury reports in the following season.

We constructed a dataset by combining two public sources: pre-scraped NBA injury transaction records from 2013 to 2018 and official player statistics retrieved through the NBA's API. After cleaning, merging, and engineering features, the final dataset contained 2,032 player-season records described by 15 features spanning workload metrics (minutes, games played, distance traveled), physical attributes (age, height, weight), prior injury history, team scheduling context (back-to-back games), and interaction terms capturing risk factors such as age combined with minutes played. The prediction target is the number of injury report appearances in the following season, which we framed as a regression task.

For supervised learning, we trained and evaluated three model families: Linear Regression, Random Forest, and XGBoost, each using five-fold cross-validation with hyperparameter tuning. We compared all results against a mean-prediction baseline. For unsupervised learning, we applied K-Means and Hierarchical Clustering to identify groupings among players based on their statistical and physical profiles (archetypes).

Our supervised results show that XGBoost performed best with a test MAE of 0.941, a slight improvement over the baseline MAE of 1.048. All models struggled to outperform the baseline, confirming that injury prediction carries a weak signal at the individual player level. Prior injury history was shown to be the single most important predictor. On the unsupervised side, clustering revealed two broad player archetypes: high-usage starters and low-usage role players, with the former group showing higher injury rates. However, our silhouette scores remained low, suggesting that player profiles exist along a continuum rather than being in specific risk categories.

Related work

When researching related projects to our NBA Injury Risk Prediction topic, we found several studies have explored injury prediction and analysis in the NBA using machine learning. Cohan et al. (2021) developed METIC, a deep learning model that forecasted NBA injuries using injury transaction data from Pro Sports Transactions, the same source we use in this project. Their approach used feature learning to automatically discover interaction patterns between variables, and framed injury prediction as a binary classification task. Our project differs by using interpretable traditional ML models (Linear Regression, Random Forest, XGBoost) and framing the problem as regression, predicting the count/incidence of injury report appearances rather than a simple yes/no outcome. We also incorporate niche NBA player tracking statistics such as distance traveled and usage rate, which their study did not include.

We also found a study published in the Orthopaedic Journal of Sports Medicine (2022) that used machine learning to predict lower extremity muscle strains specifically in NBA players from 1999 to 2019. Their XGBoost model outperformed logistic regression, and they identified prior injury history as a key risk factor, which aligns with our own findings. However, their scope was limited to one specific injury type. Our project aims to predict injury report appearances across all injury types and adds an unsupervised learning component to identify player archetypes based on physical and statistical profiles.

Another study we found is Karypidis et al. (2024), who took an unsupervised approach, using DBSCAN and association rule mining to analyze NBA injury recovery durations and their economic impacts over two decades. Their focus was on understanding what happens after injuries occur, such as recovery time and the implications on salary. Our project instead aims to predict injuries before they happen, where we combine supervised prediction with unsupervised clustering to both predict risk and categorize players into injury risk archetypes.

Data Source(s)

For our project, we combine two publicly available data sources to construct a player-season level dataset for NBA injury prediction.

The first source is a pre-scraped NBA injury dataset from the elap733 GitHub repository, which contains injury transaction records originally collected from Pro Sports Transactions (prosportstransactions.com). This dataset logs individual injury events for NBA players including the date, player name, team, and injury description. We used injury records spanning the 2013-14 through 2018-19 NBA seasons. After filtering to player injuries only (excluding rest days and non-injury transactions) and aggregating to

the player-season level, this produced 1,503 injury records. The data was provided in CSV format.

The second source is the official NBA statistics platform, accessed through the nba_api Python package. We pulled four categories of data for each season in our range: traditional box score statistics (games played, minutes, points, assists, etc.), player bio information (age, height, weight, draft year), niche player tracking data (distance traveled, average speed), and team schedule data used to calculate back-to-back game counts. Each category was retrieved via specific API endpoints and saved as individual season-level CSV files. After cleaning and combining across seasons, each category contained 3,006 player-season records covering the 2013-14 through 2018-19 seasons.

These two sources were merged by player and season using a player ID mapping process that achieved a 99.2% match rate (602 unique players) through a combination of exact name matching and fuzzy matching techniques. Fuzzy matching was necessary because player names across the two sources often differed due to spelling variations and inconsistent official name formats (e.g., nicknames vs. legal names), which prevented the exact matching for a subset of players. The final merged dataset before feature engineering contained 3,006 player-season records across 92 columns. After feature engineering and the creation of a forward-shifted prediction target, the modeling dataset contained 2,032 player-season records with 15 features, split temporally into a training set of 1,620 rows (2013-14 through 2016-17) and a test set of 412 rows (2017-18 predicting 2018-19 outcomes).

Feature Engineering

Starting from the merged dataset of 3,006 player-season records produced during EDA, we performed several transformations to create model-ready features.

We started with target variable construction, where our goal is to predict next-season injury risk, so we shifted the target variable forward by one season: each row's features describe a player's current season, while the target reflects their injury report count in the following season. For example, a player's 2015-16 statistics are used to predict their 2016-17 injury report appearances. Players in the final season (2018-19) were dropped since no future target existed, reducing the dataset from 3,006 to 2,032 rows.

We moved on to the actual feature creation, where we engineered five new features informed by patterns identified during EDA.

1. First, we created *injury_report_count_last_season*, a lag feature basically capturing a player's prior-season injury history (motivated by the EDA finding that previously injured players are roughly twice as likely to appear on injury reports)

2. Second, we created **four interaction features**
    a. *Weight_x_minutes, weight_x_minutes*
    b. *B2b_x_minutes*
    c. *Age_x_minutes*
        i. These interaction features are important to capture compound risk factors. For example, neither age nor minutes alone strongly predicts injury, but a 34-year-old playing 35 minutes per game carries a different risk profile than a 34-year-old playing 8 minutes.

Finally, our feature selection we selected 15 final features (from 92 columns of our merged dataset) across five categories:

Features (15)

Category	Features
Workload (5)	min, gp, dist_miles, usg_pct, ts_pct
Physical (3)	age, player_height_inches, player_weight
Injury history (2)	injured_last_season, injury_report_count_last_season
Team context (1)	b2b_games
Interactions (4)	age_x_minutes, weight_x_minutes, b2b_x_minutes, age_x_weight

Figure 1 - Features

Our preprocessing included imputing missing values using training set medians, primarily from players with incomplete tracking data in earlier seasons. The dataset was split temporally: seasons 2013-14 through 2016-17 formed the training set (1,620 rows) and 2017-18 formed the test set (412 rows), with the test set predicting 2018-19 injury outcomes. This temporal split mirrors real-world usage where we would only have past data available to make future predictions.

Supervised Learning

Methods Description

Our supervised learning task is to predict a player's injury report count for the following season as a regression problem. We trained three model families: Linear Regression, Random Forest, and XGBoost. All models were trained on the same 15 features and evaluated using five-fold cross-validation on the training set (1,620 rows, seasons

2013-14 through 2016-17), with final performance measured on a held-out test set (412 rows, 2017-18 features predicting 2018-19 outcomes).

We chose MAE as our primary metric because it directly represents the average prediction error in injury report counts. We also report RMSE and R^2. We established a baseline using a mean-prediction model which always predicts the training set mean of 1.413 injury reports — any useful model must outperform this. Linear Regression was included as the simplest model to establish a performance floor. Random Forest and XGBoost were selected for their ability to capture non-linear relationships and feature interactions, which EDA suggested might matter given the weak individual feature correlations (max r = 0.30).

Hyperparameter tuning was performed via GridSearchCV with five-fold cross-validation. For Random Forest, we tuned n_estimators, max_depth, and min_samples_leaf across 36 combinations. For XGBoost, we tuned n_estimators, max_depth, learning_rate, and subsample across 81 combinations. Both tree-based models required strong regularization to avoid overfitting, with optimal configurations favoring shallow trees (RF: max_depth=5, XGBoost: max_depth=2).

Supervised Evaluation

Overall Results:

The results table below summarizes performance across all model families. XGBoost achieved the best test MAE of 0.941, a 10.2% improvement over the baseline MAE of 1.048. Linear Regression performed surprisingly well (MAE 0.928), while Random Forest fell in between (MAE 0.964). All CV MAE values were consistent across folds with standard deviations around 0.109–0.113, indicating stable generalization. All models only modestly outperform the baseline, confirming that injury prediction carries inherently weak signals at the individual player level.

Model	CV MAE (mean ± std)	Test MAE	Test RMSE	Test R^2
Baseline (Mean)	N/A	1.048	1.250	-0.032
Linear Regression	1.158 ± 0.109	0.928	1.172	0.093
Random Forest (tuned)	1.142 ± 0.109	0.964	1.211	0.033
XGBoost (tuned)	1.130 ± 0.113	0.941	1.189	0.067

Figure 2 - Performance Across Model Families

Feature Importance and Ablation Analysis:

The figures below show XGBoost's feature importance scores. The interaction term age_x_minutes ranked highest, followed by weight_x_minutes and games played. We validated these rankings through ablation analysis by removing one feature group at a time and retraining. Removing Injury History increased MAE by 0.015, confirming prior injury as the most critical predictor. Team Context and Interaction features each increased MAE by 0.013. Surprisingly, removing all five Workload features actually improved MAE by 0.016, suggesting these features introduce more noise than signal once injury history is accounted for.

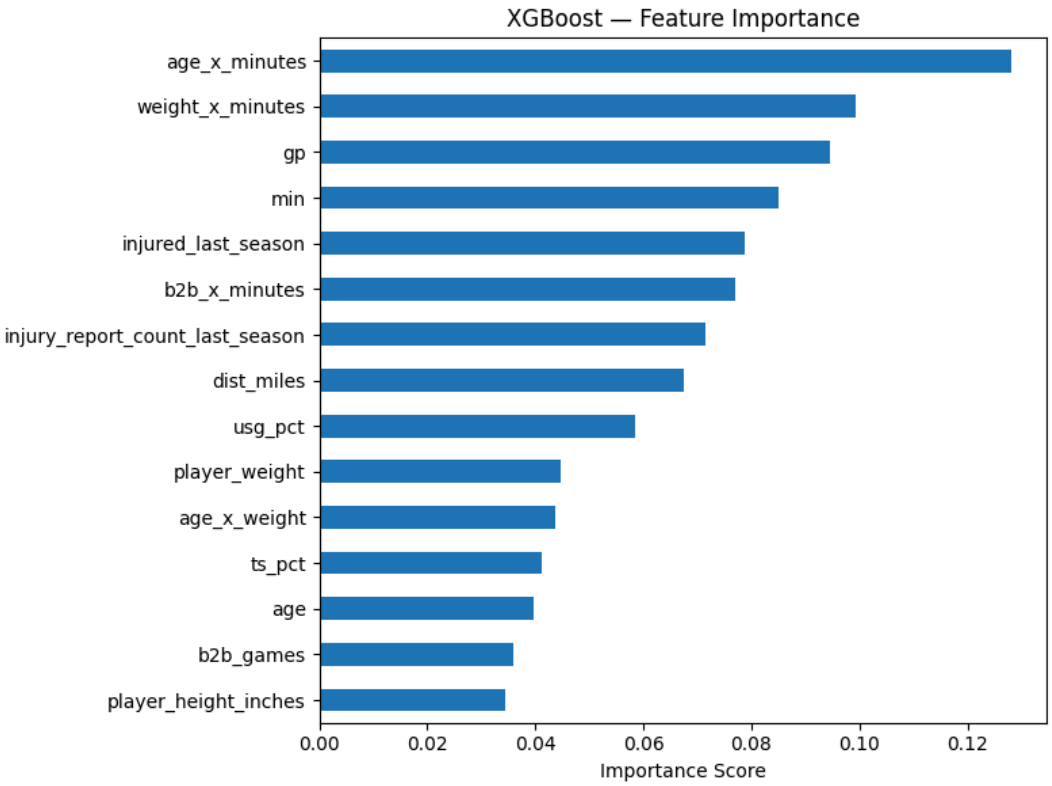

Figure 3 - XGBoost - Feature Importance

Removed Group	Features Removed	MAE Without	MAE Impact
Injury History	2	0.956	0.015
Team Context	1	0.954	0.013
Interactions	4	0.954	0.013
Physical	3	0.948	0.007
Workload	5	0.925	-0.016

Figure 4 - XGBoost - Feature Importance Scores

Sensitivity Analysis:

The figure below shows how XGBoost's test MAE changes as each hyperparameter varies from its optimal value. The model is most sensitive to max_depth — increasing from 2 to 7 raises MAE by about 0.05, confirming that deeper trees overfit the weak injury signal. Learning rate has a sweet spot around 0.05–0.1. The model is relatively stable across n_estimators and subsample values.

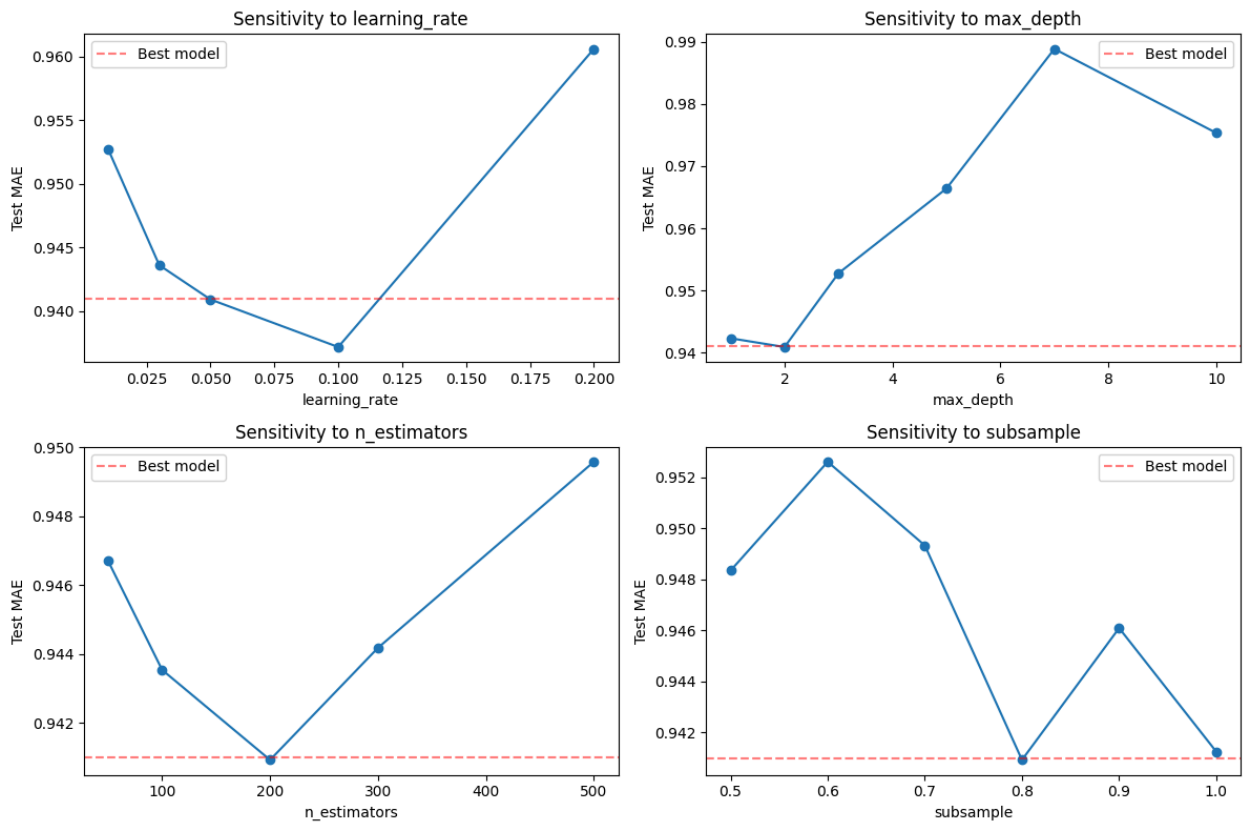

Figure 5 - Sensitivity Analysis Graphs

Tradeoffs:

The main tradeoff we observe is model complexity versus generalization. Default configurations of both Random Forest and XGBoost performed worse than Linear Regression, only surpassing it after aggressive regularization. For weak-signal tasks like injury prediction, simpler models may be preferable unless overfitting is carefully controlled. We also see a tradeoff between predicting the majority well (zero-injury players) versus capturing rare high-injury seasons.

Failure Analysis:

We examined the ten worst predictions from XGBoost, all of which were underpredictions of high-injury seasons. We categorized these into three types.

1. **Category 1: Sudden Spike (2 cases):** young, healthy players with zero injury history who suddenly had 4 injury reports. The model has no signal to predict these.
2. **Category 2: Severity Underestimated (4 cases):** players with 3–4 prior injuries who had 5–7 the next season. The model knows they're at-risk but predicts around 2 instead of 5+.
3. **Category 3: Moderate Undershoot (4 cases):** some injury history, target of 4, model predicts around 1.2. The root cause is the zero-inflated target (50.7% zeros), which pushes the model toward conservative predictions. Possible improvements include adding injury-type features, multi-season workload trends, or reframing as a binary classification problem.

```
=== Failure Category 1: Healthy profile, sudden spike ===
Players with NO prior injury history who got 4+ injuries next season
Count: 2
 min  gp  age  injured_last_season  target_next_season  predicted     error
12.3  20 21.0                  0.0                   4   0.529381 -3.470619
23.2  63 23.0                  0.0                   4   1.231610 -2.768390

=== Failure Category 2: Known risk, but severity underestimated ===
Players WITH prior injuries, but actual outcome far exceeded prediction
Count: 4
 min  gp  age  injury_report_count_last_season  target_next_season  predicted     error
36.4  75 25.0                              3.0                   7   2.088427 -4.911573
18.3  81 31.0                              3.0                   5   0.990270 -4.009730
16.8  25 29.0                              4.0                   5   1.012937 -3.987063
29.9  73 31.0                              3.0                   5   2.046504 -2.953496

=== Failure Category 3: Moderate history, moderate undershoot ===
Prior injuries, target = 4, model predicted ~1
Count: 4
 min  gp  age  injury_report_count_last_season  target_next_season  predicted     error
12.1  73 24.0                              1.0                   4   0.815591 -3.184409
14.0  53 30.0                              3.0                   4   1.211215 -2.788785
14.4  32 28.0                              3.0                   4   1.212435 -2.787565
33.2  81 24.0                              2.0                   4   1.221217 -2.778783
```

Figure 6 - XGBoost - Failure Category

Unsupervised Learning

In the unsupervised learning portion, we used the same processed dataset from the output of the feature engineering to attempt to cluster players into injury risk archetypes such as durable young players, high-mileage veterans, or injury-prone stars. We applied and explored two different unsupervised methods: K-means and Hierarchical clustering.

The first unsupervised method used was K-Means clustering. We selected K-Means because it is fast, centroid-focused, and uses partitional clustering. K-means is very interpretable and understandable compared to other clustering techniques, where each point is assigned to the cluster with the closest centroid. This makes it relatively easy to understand and accessible for quicker data analysis and visualization. One of the potential drawbacks of K-means clustering is that it requires an initial k value, which is a predetermined number of groups. This is addressed in our hyperparameter tuning/exploration for this model. The two hyperparameters for this model that we focused on are the k value (n_clusters) and the n_init value. The n_init value is the number of times k-means runs with different random initial centroid seeds, then the best output of those runs is selected based on the inertia. This is another important hyperparameter to use as different initializations can result in different solutions. We looped through the K-means clustering using k values from 2 to 10 and a n_init value of 20 iterations. From this iteration we selected the ideal clusters by using silhouette scores and inertia measurements of each output.

The second unsupervised method explored was Hierarchical Clustering to examine groupings at different similarity levels. Specifically, we selected Agglomerative clustering for our analysis. We selected this model for its simplicity, interpretability, and the lack of need to define a predetermined k value. We used the same features from the output of feature engineering, the same as K-means. Initially, we generate a hierarchical clustering dendrogram which shows the sequences of merging / splitting during each iteration. From there, the hyperparameters of focus are the k (n_clusters) and the linkage type. We loop through the linkage types and k values to determine if the dataset exhibits any continuous structure rather than a discrete cluster separation.

Cluster quality was graded using silhouette scores and inertia for each cluster combination, and we determined whether clusters differ in actual games missed. Silhouette scores measure cluster cohesion (distance within own cluster) and separation (distance from other clusters). Inertia focuses on how tight the clusters are and is mainly used in the elbow plots. Visualizations will include elbow plots, silhouette scores vs k, and PCA scatter plots showing cluster separation.

The best models from K-means and Hierarchical both have k values of 2, and the hierarchical model linkage type of average. In K-means the elbow plot and silhouette score plot suggest the dataset exhibits weak cluster structure. In the elbow plot, the inertia decreases gradually without a clear inflection point, indicating the absence of a well-defined “elbow” and no obvious optimal value of k. Similarly, the silhouette scores peak at k = 2, but the maximum value of 0.23 is relatively low.

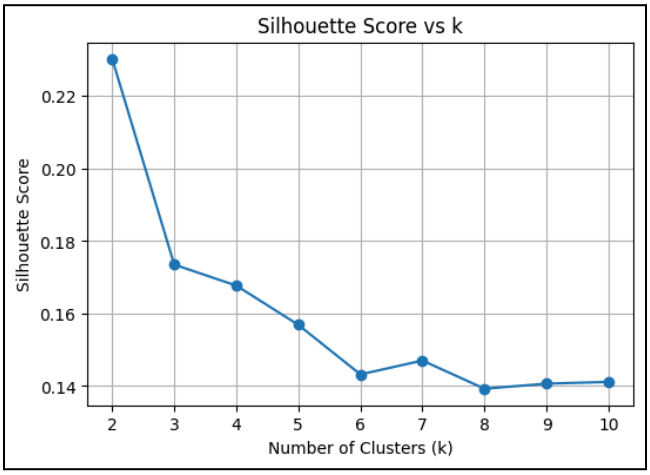

Figure 7 - Silhouette Score vs k for K-means

This suggests that even at the best-performing cluster count, the separation between clusters is weak and there is substantial overlap among data points. This can be visually explained through the PCA plots of all k below - clearly there are no distinct clusters seen with clear overlap of cluster edges.

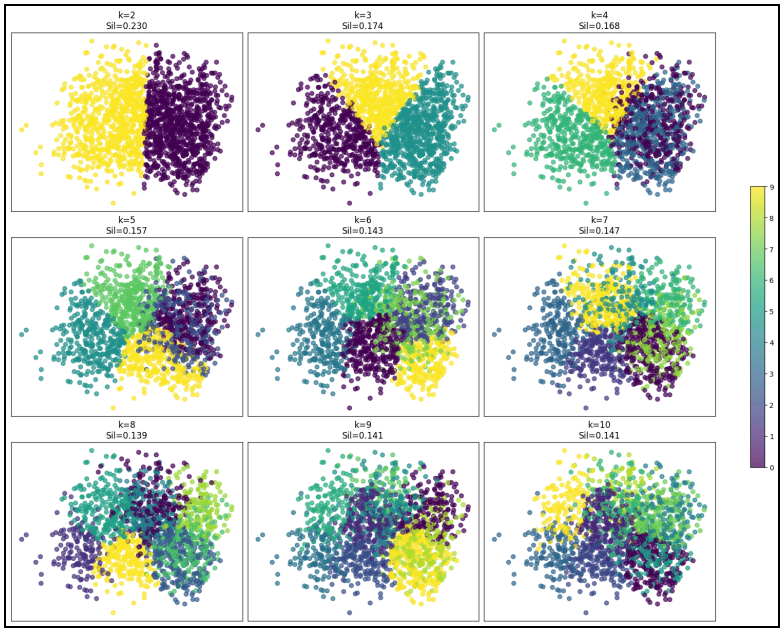

Figure 8 - PCA Plots from K-means

In the hierarchical clustering, the most optimal clustering hyperparameter combination was at k=2 and using the average linkage method with a silhouette score of 0.47. The below silhouette plot vs. k shows the differences between k values and linkage types with the average (green) line with the highest silhouette score at k=2. Silhouette scores drop as k increases for all linkage types. One main difference between hierarchical and k-means is that we removed the multiplication features (such as age_x_minutes). We observed higher silhouette clustering scores without these features in this model while K-means showed the opposite. Despite average and complete linkage having higher clustering scores, the cluster breakdown themselves were very lopsided. At k=2 and average linkage, the cluster sizes were 1617 and 3.

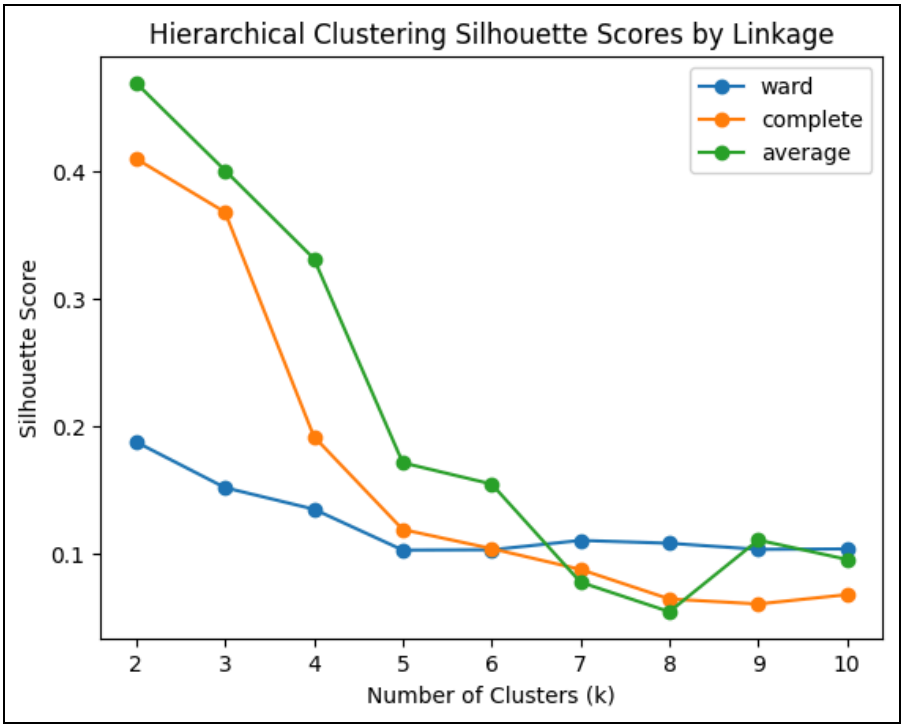

Figure 9 - Hierarchical Clustering Silhouette Scores

In the figure below, an output dendrogram plot of the clustering is shown for the average linkage method. The short vertical lines between the merging of clusters suggest that the clusters are similar to each other. If there were stronger clustering characteristics amongst the data, we would see much taller horizontal lines.

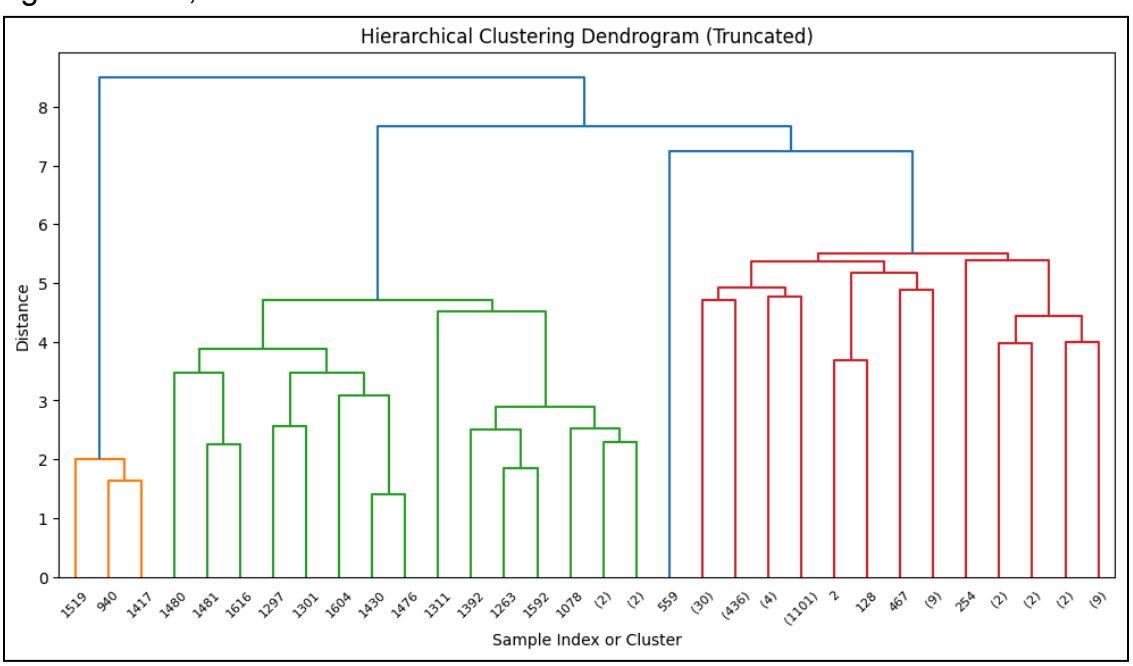

Figure 10 - Hierarchical Clustering Dendrogram

We conducted a sensitivity analysis on how the hyperparameter and feature choices affected the model results. For hyperparameters we ran the clustering with k values from 2-10. We ran this twice with and without the multiplied features to compare how sensitive the silhouette scores would be.

```
--- Hierarchical Clustering: Sensitivity to Linkage Method WITHOUT Mulitplied Features ---
k             2      3      4      5      6      7      8      9      10
Linkage
average   0.470  0.401  0.332  0.172  0.155  0.078  0.055  0.111  0.096
complete  0.410  0.368  0.192  0.119  0.104  0.088  0.065  0.061  0.068
ward      0.188  0.152  0.135  0.103  0.103  0.111  0.108  0.104  0.104
```

```
--- Hierarchical Clustering: Sensitivity to Linkage Method WITH Multiplied Features ---
k             2      3      4      5      6      7      8      9      10
Linkage
average   0.422  0.392  0.273  0.233  0.198  0.210  0.205  0.190  0.159
complete  0.415  0.362  0.233  0.198  0.149  0.126  0.117  0.089  0.085
ward      0.249  0.185  0.134  0.128  0.131  0.110  0.111  0.119  0.125
```

Figure 11 - Hierarchical Clustering: Sensitivity Analysis

The sensitivity results show that for the average linkage type, scores are higher for small clusters until k=5. Removing the multiplied features really only showed higher scores for smaller clusters while higher scores occurred for larger clusters when these features were included. Although there are some relative differences, the silhouette scores are still relatively low and close to 0 which suggests weak clustering in the dataset.

Based on the results of both K-means and Hierarchal clustering, the current dataset and features suggest that players cannot be easily naturally clustered together. Instead, the injury-related features appear to vary along a continuum rather than forming discrete archetypes. The best clustering was at k=2 using the K-means dataset which differentiated ‘high usage’ players such as NBA starters (cluster 0) versus ‘low usage’ players such as bench players or ones playing sparingly (cluster 1). Despite some hierarchical clusters having better scores, the clusters were extremely imbalanced and not feasible. A comparison of the target variable between the two clusters shows a difference in injury percentage.

```
--- Injury Outcomes by Cluster ---
         Mean Injuries  Median Injuries    Std  Count
cluster
0                1.762              1.0  1.658    911
1                0.965              1.0  1.283    709

% of players with at least 1 injury report:
  Cluster 0 (high-usage): 72.2%
  Cluster 1 (low-usage):  51.1%
```

Figure 12 - Injury Outcomes by Cluster

Discussion

From Part A (Supervised Learning), we learned that predicting NBA injury report counts is significantly more difficult than expected. Although we engineered interaction features and tuned multiple models, all supervised models only modestly outperformed the baseline. XGBoost achieved a test MAE of 0.941 compared to the baseline MAE of 1.048, representing improvement but confirming that injury prediction contains a weak signal at the individual player level. One surprising result was that Linear Regression performed nearly as well as XGBoost, suggesting that the predictive structure in the data is relatively simple once prior injury history is included. We were also surprised that removing workload features slightly improved performance during ablation analysis, indicating that once prior injury history is accounted for, some workload variables may introduce noise rather than additional signal.

A major challenge in Part A was the zero-inflated target variable, where over half of player-seasons contained zero injury reports. This distribution pushed models toward conservative predictions and led to systematic underprediction of high-injury seasons, as shown in our failure analysis. Rare but extreme injury spikes were especially difficult to anticipate because the model lacked strong prior signals. To respond to these challenges, we used temporal train-test splits to mirror real-world deployment, applied strong regularization to prevent overfitting, and conducted ablation and sensitivity analyses to better understand model behavior. With additional time and resources, the solution could be extended by reframing the problem as a classification task, applying zero-inflated count models, incorporating multi-season workload trends, or adding more granular injury-type information. These improvements may better capture rare high-injury events that regression models struggled to predict.

From Part B (Unsupervised Learning), we learned that players do not naturally form strong, discrete injury risk archetypes. K-Means clustering produced low silhouette scores, with a maximum around 0.23, indicating weak separation and substantial overlap. Even hierarchical clustering, which performed somewhat better in sensitivity

analysis, still showed that clusters were not strongly distinct along with extremely imbalanced cluster sizes. This suggests that injury-related characteristics exist along a continuum rather than in clear categories. Although the best-performing configuration at k=2 separated high-usage starters from low-usage role players, the separation was primarily driven by workload and prior injury history, aligning closely with the supervised learning findings.

The main challenge in Part B was determining whether meaningful cluster structure existed at all. Through systematic hyperparameter tuning, linkage comparisons, and feature sensitivity analysis, we ensured that clusters were evaluated rigorously rather than assumed. With additional time, future work could explore density-based clustering methods, nonlinear dimensionality reduction techniques, longitudinal career features, or clustering in learned latent feature spaces to better detect subtle structure in the data.

Ethical Considerations

Several ethical concerns arise from the supervised injury prediction model. First, labeling players as “high risk” could influence contract negotiations, playing time decisions, or public perception, especially given that predictive performance only modestly exceeds baseline and remains uncertain. Overconfidence in such a model could unfairly disadvantage players. Second, because prior injury history is the strongest predictor, the model may reinforce historical patterns and potentially amplify structural biases in injury reporting or medical treatment. Third, teams could misuse predictions for financial leverage rather than for player health protection. These concerns can be mitigated by clearly communicating model limitations, emphasizing uncertainty, conducting fairness audits across player groups, and positioning the model strictly as a decision-support tool rather than a replacement for medical judgment.

Ethical issues also arise in the unsupervised clustering component. Reducing players to simplified archetypes such as “high-usage” or “low-usage” risks oversimplifying complex health profiles, especially given the weak cluster separation observed. Presenting clusters as definitive categories could mislead stakeholders into believing that injury risk is categorical rather than continuous. To address this, cluster quality metrics should always be reported transparently, and results should be framed as exploratory patterns rather than fixed labels. Additionally, although the dataset uses publicly available information, injury data relates to medical status, so analyses should remain aggregated and avoid speculation about individual diagnoses.

Overall, the ethical responsibility of this project is to maintain humility about predictive limits, communicate uncertainty clearly, and ensure that models are used to support player health and informed decision-making rather than to penalize or stigmatize athletes.

Statement of Work

Jeremy Tang - Co-led feature engineering, led unsupervised learning, contributed towards the report.

Ashhad Jaffer - Lead EDA, Supervised. Contributed towards the report.

Naseem Heydari - Co-led supervised. Led report.

Gen AI Usage

The team used Gen AI in the coding section to assist in feature engineering development, data cleaning, and optimization of code in machine learning.

References

1. Cohan, A., Schuster, J., & Fernandez, J. (2021). A deep learning approach to injury forecasting in NBA basketball. *Journal of Sports Analytics*, 7(3), 177–190. https://doi.org/10.3233/JSA-200529
2. Haeberle, H. S., Gallo, M. C., Gans, C. V., et al. (2022). Machine Learning for Predicting Lower Extremity Muscle Strain in National Basketball Association Athletes. *Orthopaedic Journal of Sports Medicine*, 10(7). https://doi.org/10.1177/23259671221108213
3. Karypidis, E., Goudas, T., Georgiou, D., Moustakas, K., & Sarigiannidis, P. (2024). Unsupervised Learning in NBA Injury Recovery: Advanced Data Mining to Decode Recovery Durations and Economic Impacts. *Information*, 15(1), 61. https://doi.org/10.3390/info15010061
4. Swar. (n.d.). nba_api: An API Client for nba.com. GitHub. https://github.com/swar/nba_api
5. elap733. (n.d.). NBA Injury Data: Web scraper and dataset for NBA injury information. GitHub. https://github.com/elap733/NBA-Injury-Data
6. Pro Sports Transactions. (n.d.). Basketball Transactions. https://www.prosportstransactions.com